

Multi-Agent Systems

Cortinovis Nicola , Lucas Marta
Università degli studi di Trieste

Multi-Agent Reinforcement Learning on Stochastic Game: Introduction

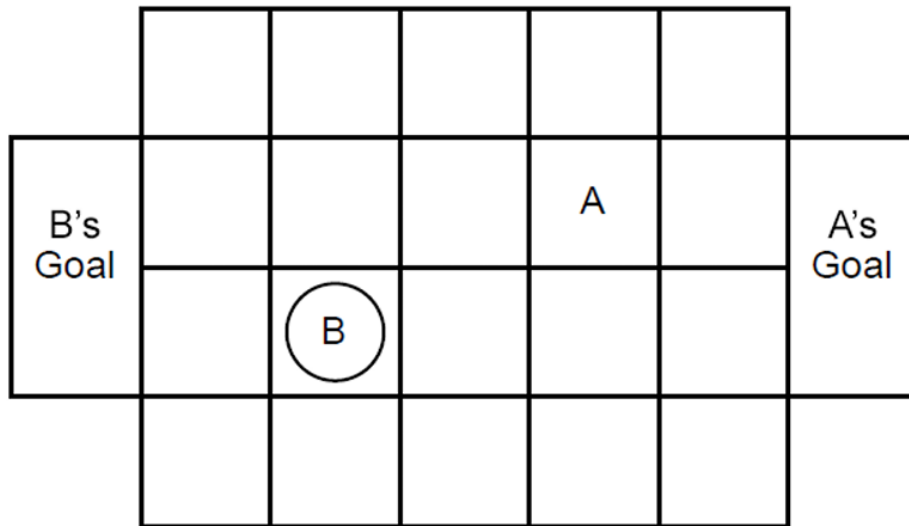
Implementation of the simplified football game by Michael L. Littman

Multi-Agent Reinforcement Learning

Two player zero-sum stochastic game

Belief-based joint action algorithm

Problem setting : Simplified Football Game



Field : 4 x 5 grid

Goalposts: size 2, outside the grid

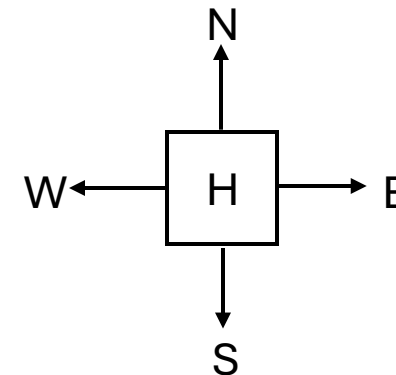
Players : A and B

Initial position A : (1,3)

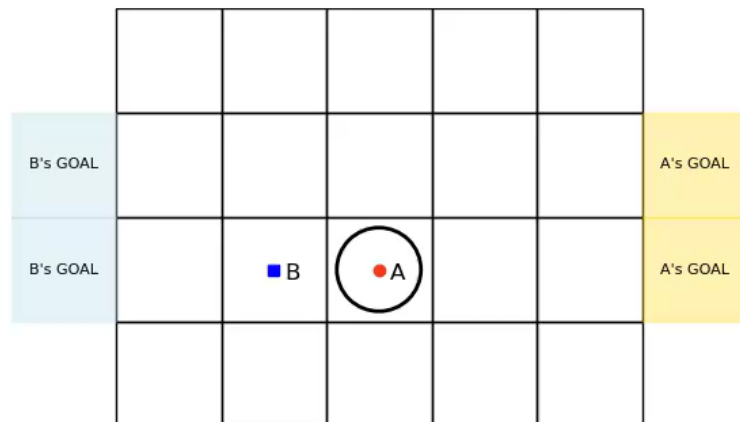
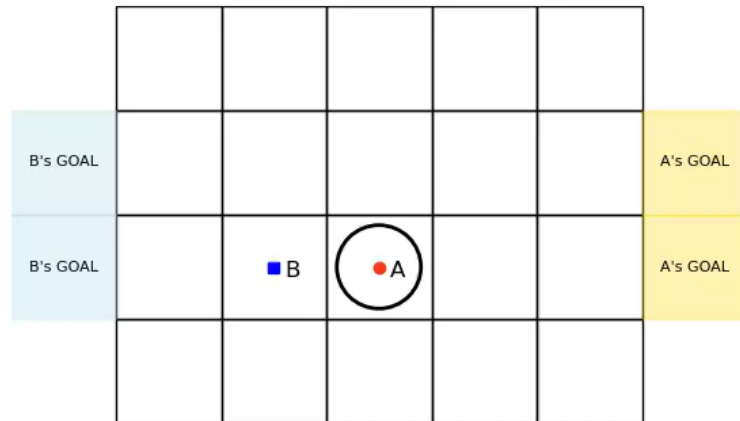
Initial position B : (2,1)

Initial ball possession: Randomly chosen

Moves: North, South, East , West, Stand (H)



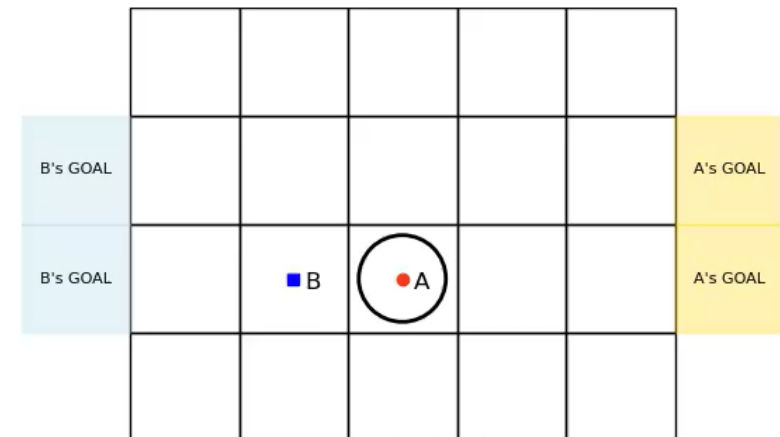
Simplified Football Game: Player collisions



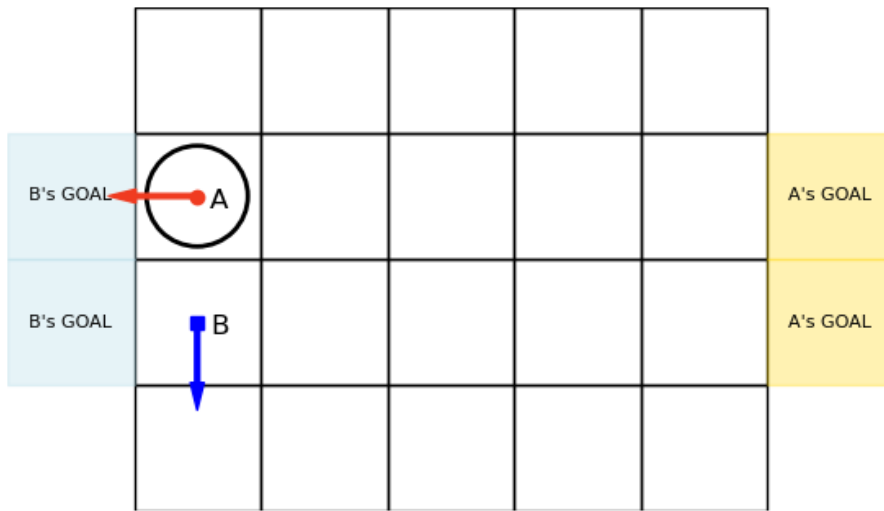
Ball possession:

If the ball carrier collides with an opponent, possession is transferred to the defender.

If both players move into each other's square, ball possession goes to the one who moves first, regardless of who had it before.



Simplified Football Game : Rewards

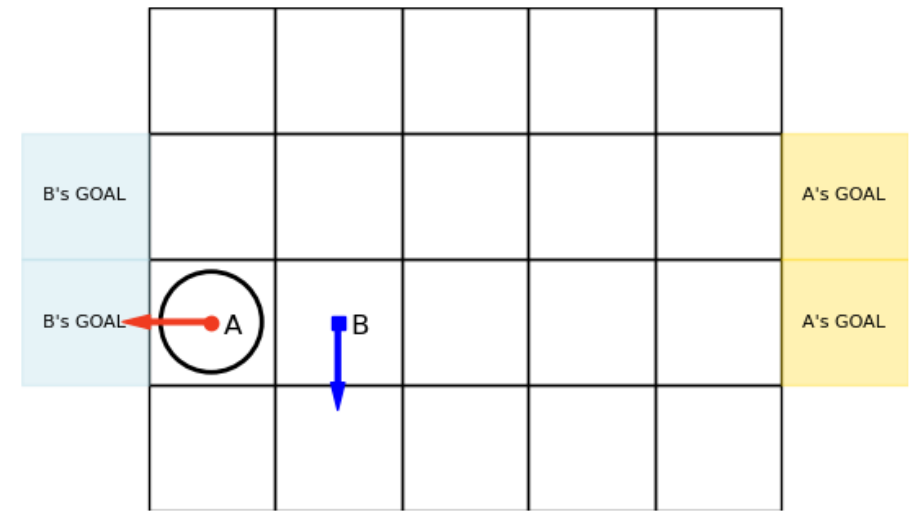


Rewards :

A scores | A: + 1, B: -1

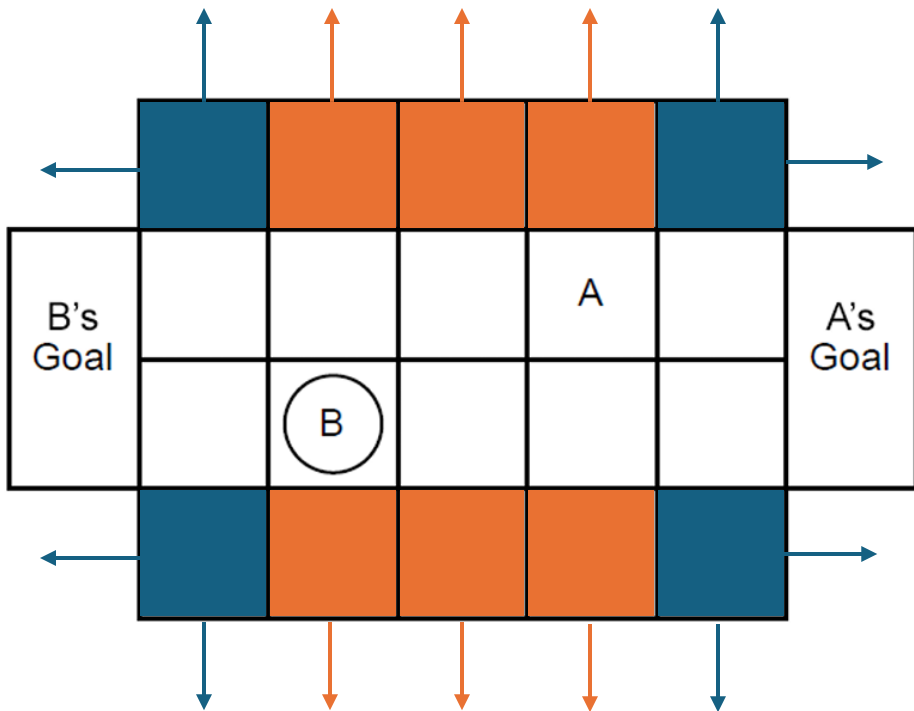
B scores | A: - 1, B: +1

Draw | A: 0, B: 0



During testing at each step there is a probability of 0.1 that games terminate prematurely in a draw

Legal actions



Actions that resulted in the opponent exiting the grid, if chosen, would impact the belief of the agent, feeding it “wrong” information



In the problematic edge cases, the choice of actions was consequently limited to **legal** ones

Belief-Based Joint Action learning algorithm

Key Points

Shared Q-table over joint actions $Q(s, a, o)$

Belief updating via counts $B_t(s)(o) = \frac{N_t(s, o)}{\sum_{o'} N_t(s, o')}$

Action-value update $Q_{t+1}(s_t, a_t, o_t) = (1 - \alpha_t) Q_t(s_t, a_t, o_t) + \alpha_t (r_t + \gamma V_t(s_{t+1}))$

State-value estimation by expectation over the belief $V_{t+1}(s) = \max_{a_1 \in \mathcal{A}_A(s)} \sum_{a_2 \in \mathcal{A}_O(s)} B_{t+1}(s)(a_2) Q_{t+1}(s, a_1, a_2)$

Experiments

Training

$$\varepsilon = 0.2 \quad \gamma = 0.9 \quad \alpha_0 = 1$$

$$\alpha_{t+1} = \alpha_t \cdot 10^{\log_{10}^{0.01/10^6}}$$

Over 10^6 steps

Belief Based Agent trained versus a random agent BBA(r)

Belief Based Agent trained versus an identical Belief Based Agent BBA(b) (self-play)

Testing

$$\gamma = 0.9$$

Probability of **no** draw at each step

Over 10^5 steps

- BBA(r) vs RA
- BBA(b) vs BBA(b)
- BBA(r) vs BBA(b)
- RA vs BBA(b)

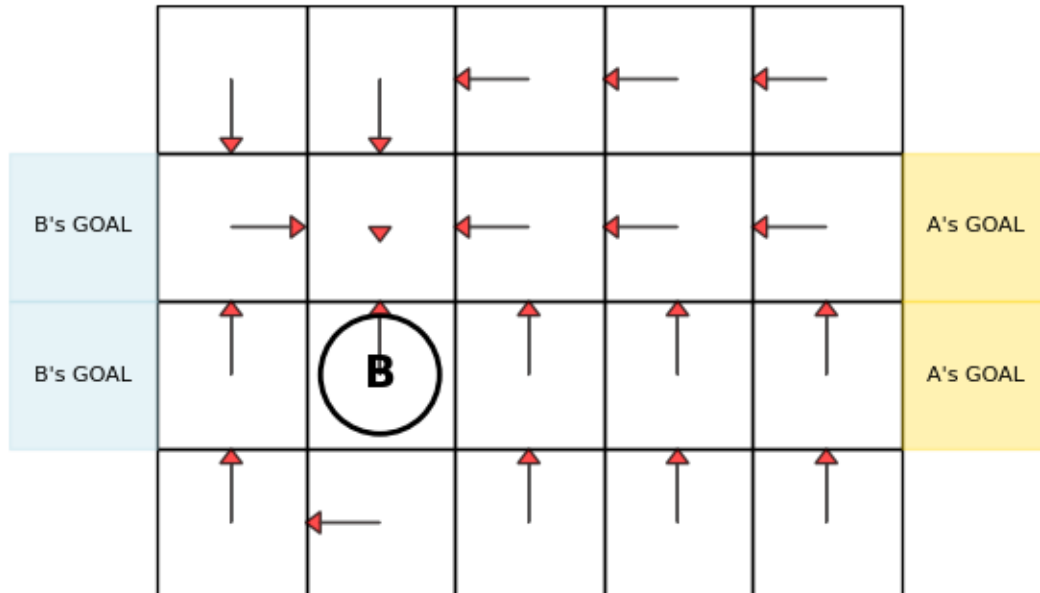
Testing results

Experiment	A Wins (%)	B Wins (%)	Draw (%)	Games	Avg Length	Std Length
<i>BBA(r) vs RA</i>	46.04	0.35	53.81	17,334	5.77	4.53
<i>BBA(b) vs BBA(b)</i>	29.33	30.03	40.23	20,945	4.78	2.68
<i>BBA(r) vs BBA(b)</i>	10.17	40.82	49.01	20,886	5.11	3.83
<i>RA vs BBA(b)</i>	0.42	45.06	54.52	16,914	5.92	4.70

Policies BBA(r) vs RA at game start setup

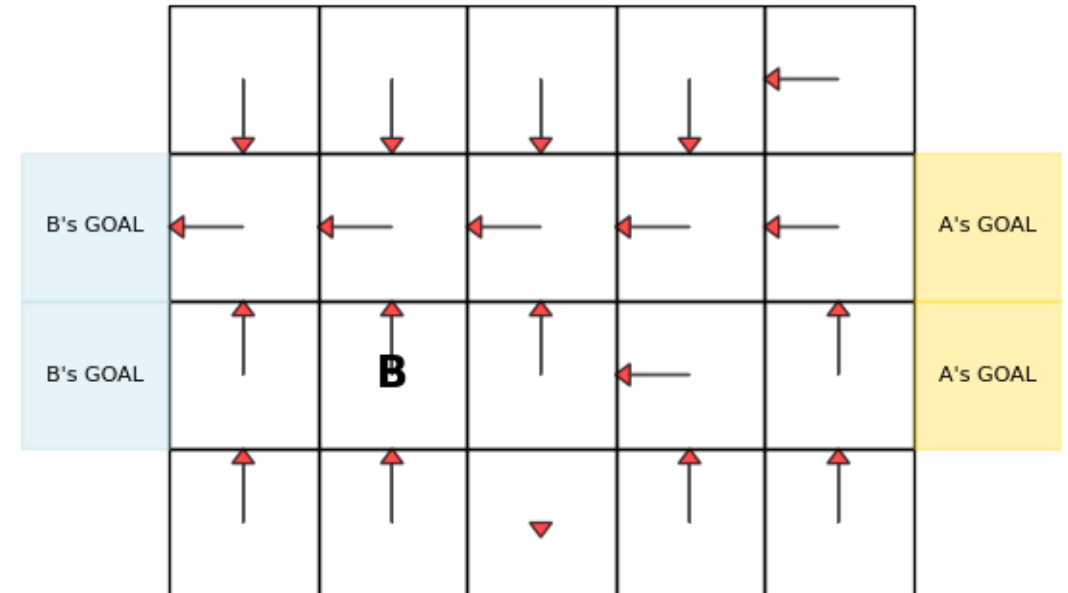
A player π without ball ownership at game start

BBA(r) vs RA



A player π with ball ownership at game start

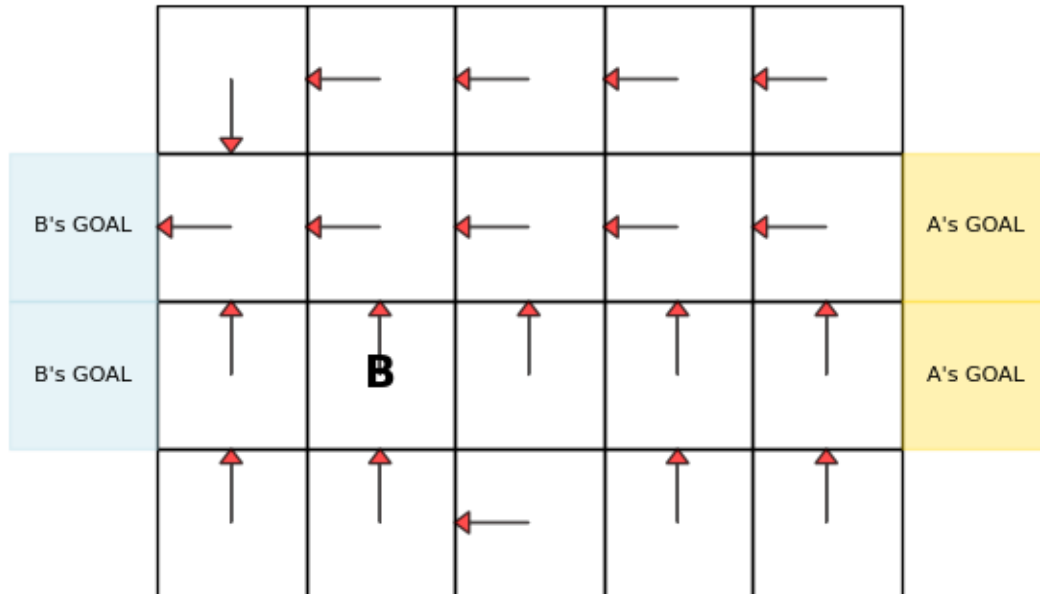
BBA(r) vs RA



Policies BBA(b) vs BBA(b) at game start setup

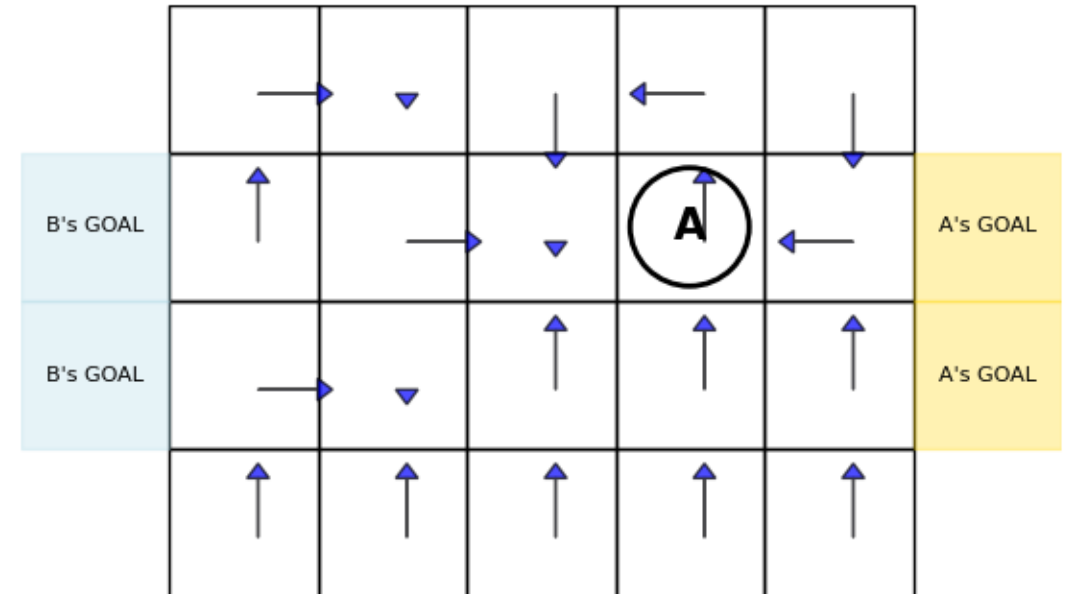
A player π with ball ownership at game start

BBA(b) vs BBA(b)

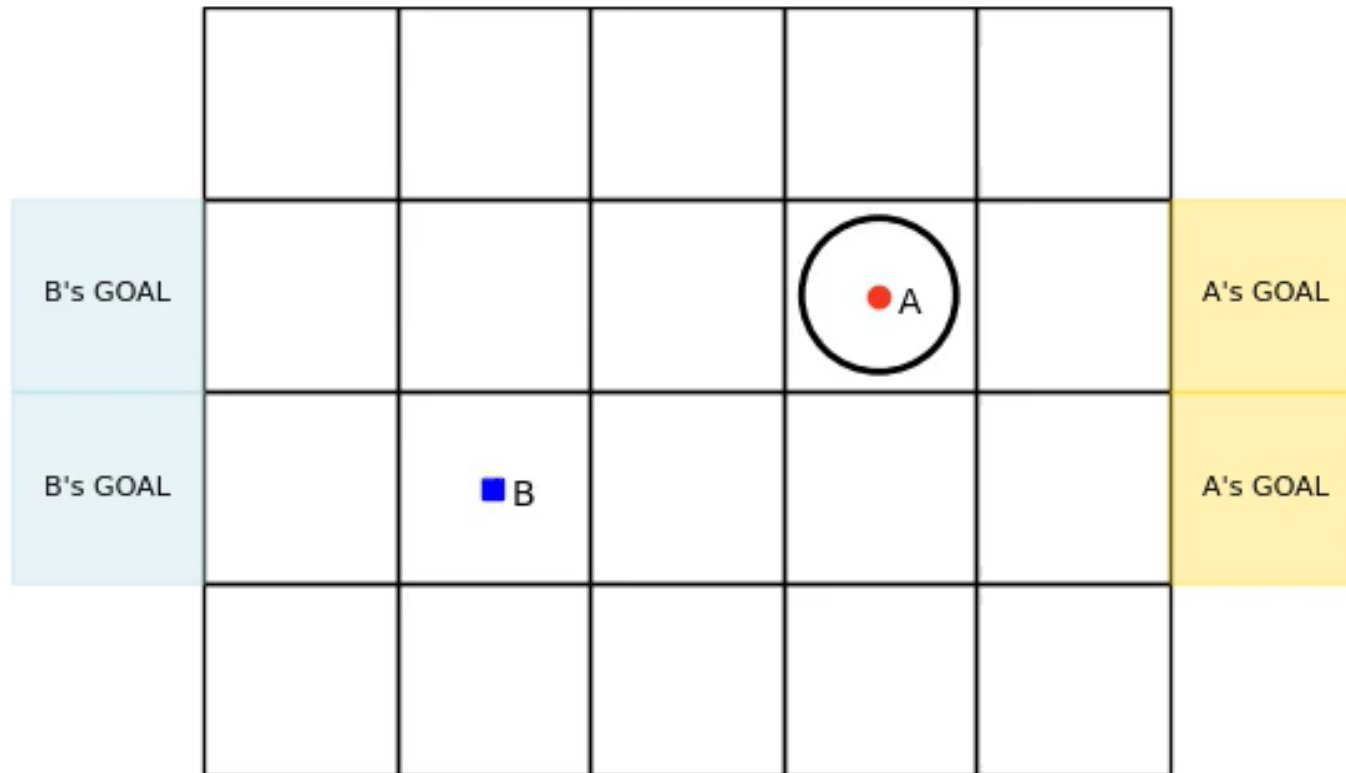


B player π without ball ownership at game start

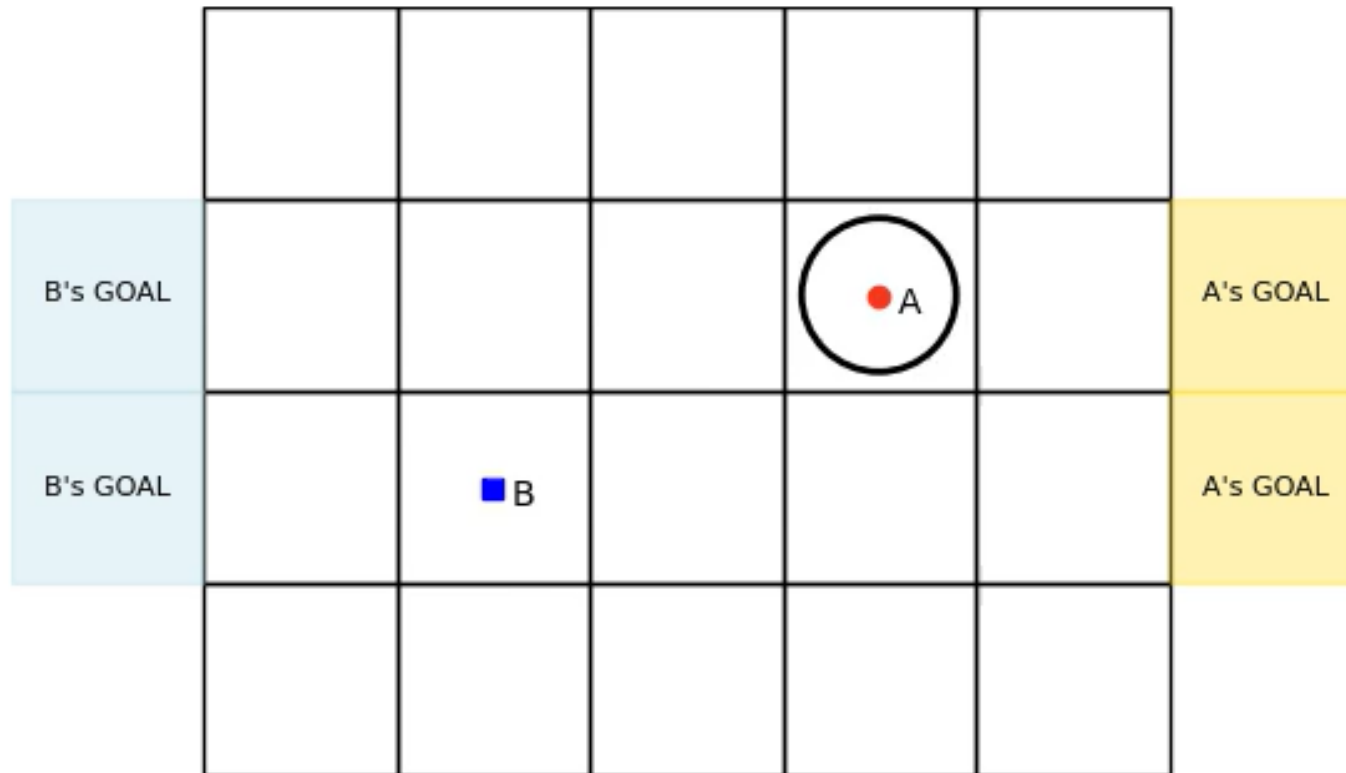
BBA(b) vs BBA(b)



Episode BBA(r) vs RA



Episode BBA(b) vs BBA(b)



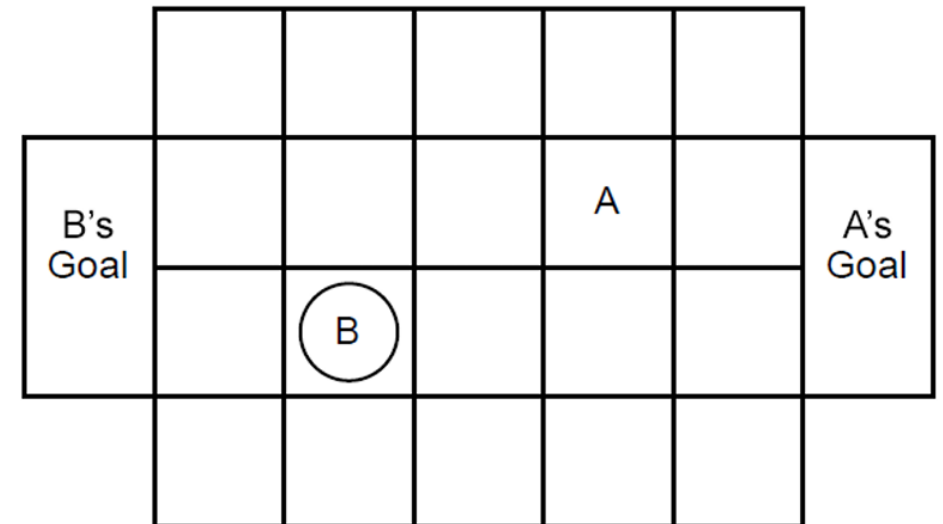
Conclusion

The observed results are consistent with what we expected from the Littman paper.

The RA got easily beaten by both BBA(r) and BBA(b)

The matches between the two BBA(b) show a balance between the win rates, which is expected since they are identical and have been trained together (self play)

Matches between BBA(r) and BBA(b) highlight that the latter learned to stop the “rush” strategy used by BBA(r) to exploit the random agent.



References

- Markov games as a framework for multi-agent reinforcement learning, Michael L. Littman, 1994
- Project code: github.com/NicolaCortinovis/MAS_2025

Scheduling Problem with Random Arrivals Introduction

Implementation of the scheduling problem with random arrivals

Poisson process to model job arrival, exponential and log-normal distribution to model job duration

Comparison of multiple scheduling strategies

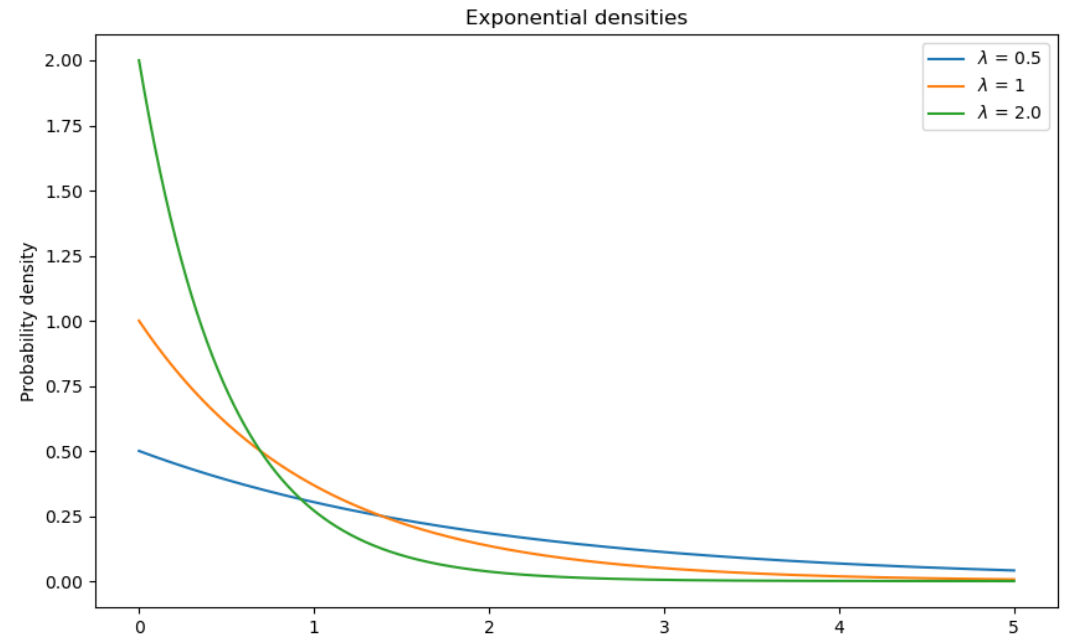
Evaluation under varying load conditions and analysis of different performance metrics

Exponential Distribution

$$X \sim \text{Exp}(\lambda)$$

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0. \\ 0 & x < 0. \end{cases}$$

$$\mathbf{E}[X] = \frac{1}{\lambda} \quad \mathbf{Var}(X) = \frac{1}{\lambda^2}$$



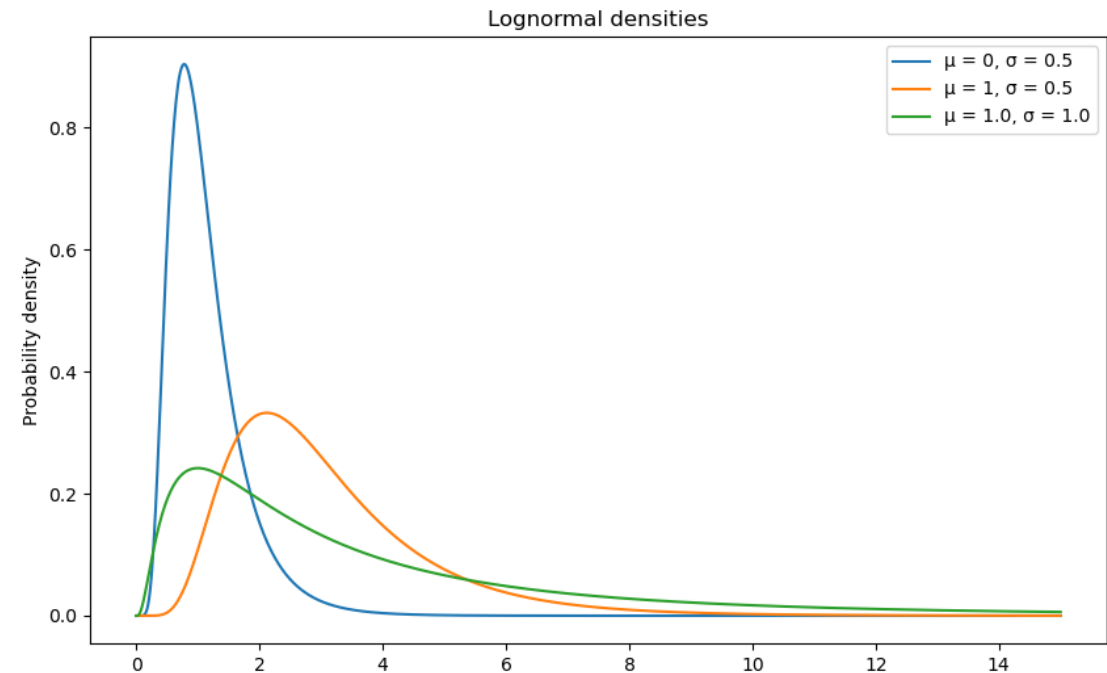
Log-Normal Distribution

$$X \sim \mathcal{N}(\mu, \sigma) \quad Y \sim e^X \quad \text{Lognormal}(\mu, \sigma)$$

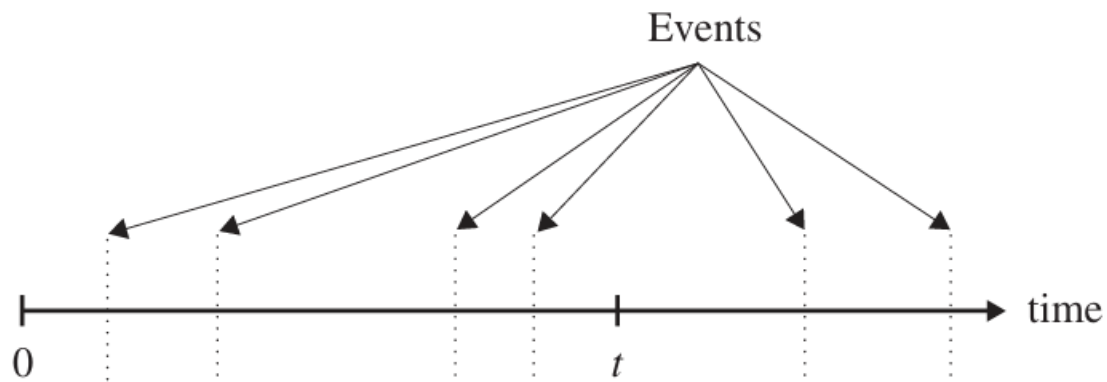
$$\log(Y) = X \sim \mathcal{N}(\mu, \sigma)$$

$$f(x) = \frac{1}{x\sigma\sqrt{2\pi}} \exp\left(-\frac{(\ln x - \mu)^2}{2\sigma^2}\right), \quad x > 0$$

$$\mathbb{E}[X] = e^{\mu + \frac{\sigma^2}{2}} \quad \text{Var}(X) = (e^{\sigma^2} - 1) e^{2\mu + \sigma^2}$$



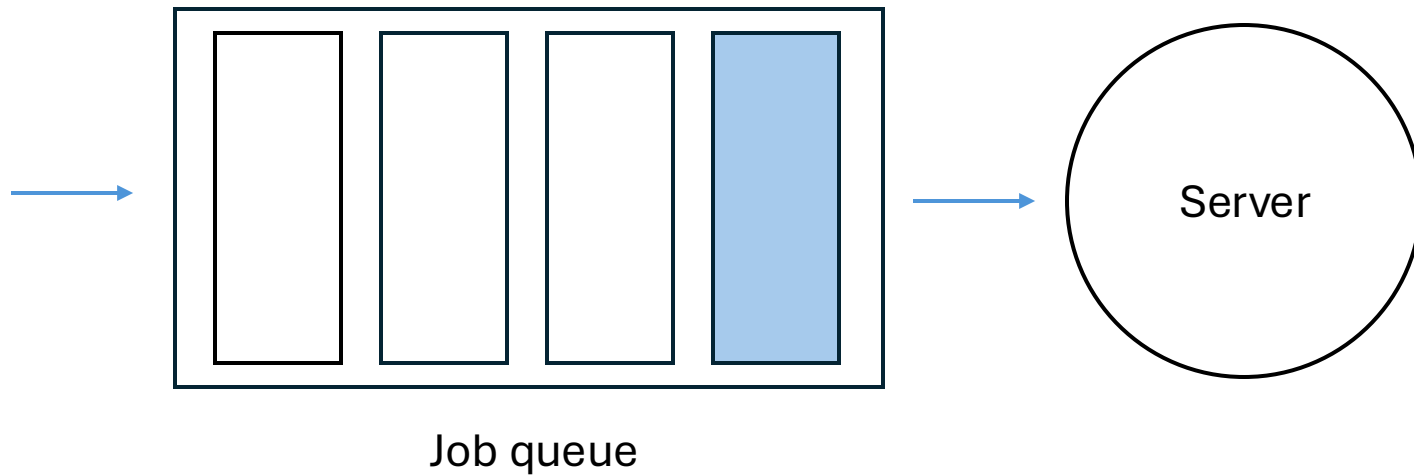
Poisson Process



$N(t), t \geq 0$ Is the number of events that occurred by time t

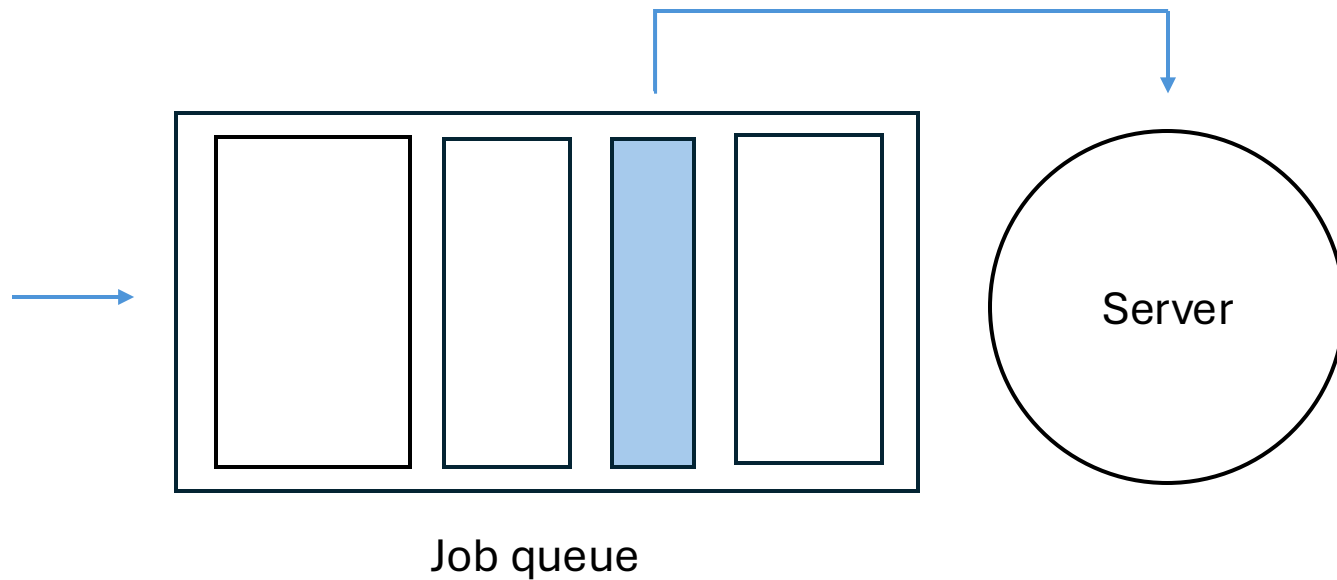
A Poisson process with rate λ is a sequence of events such that the **interarrival times** are i.i.d. Exponential random variables with rate λ and $N(0) = 0$.

First In First Out (FIFO)



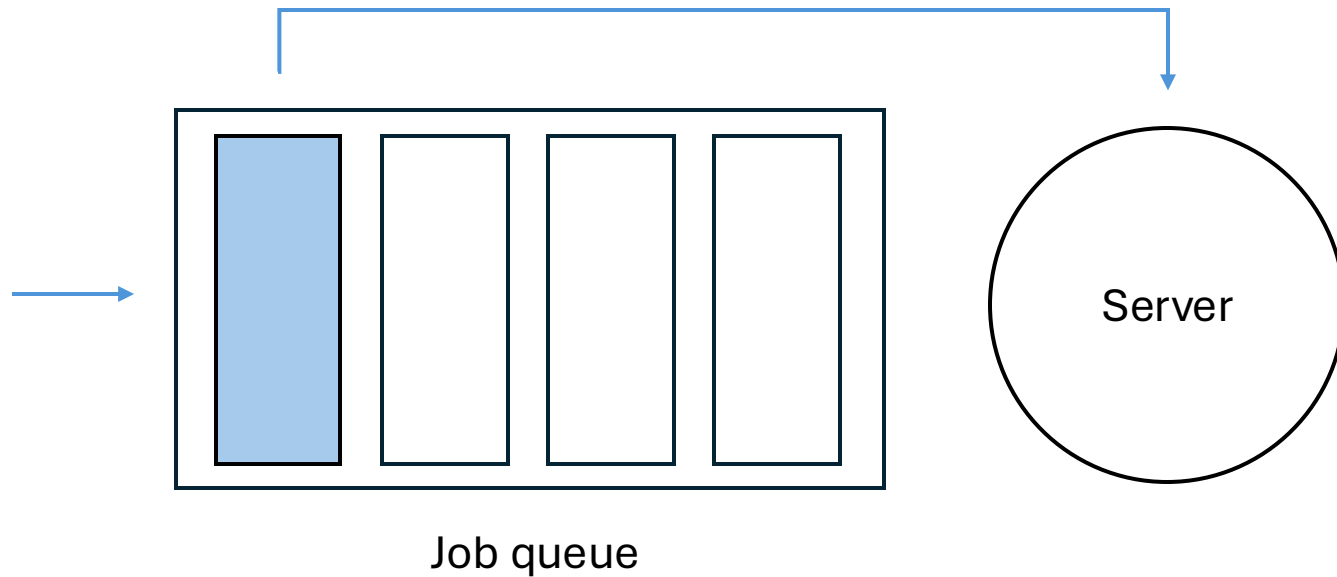
A non-preemptive scheduling strategy that gives priority to jobs based on their arrival order, favoring those which arrive first.

Shortest Job First (SJF)



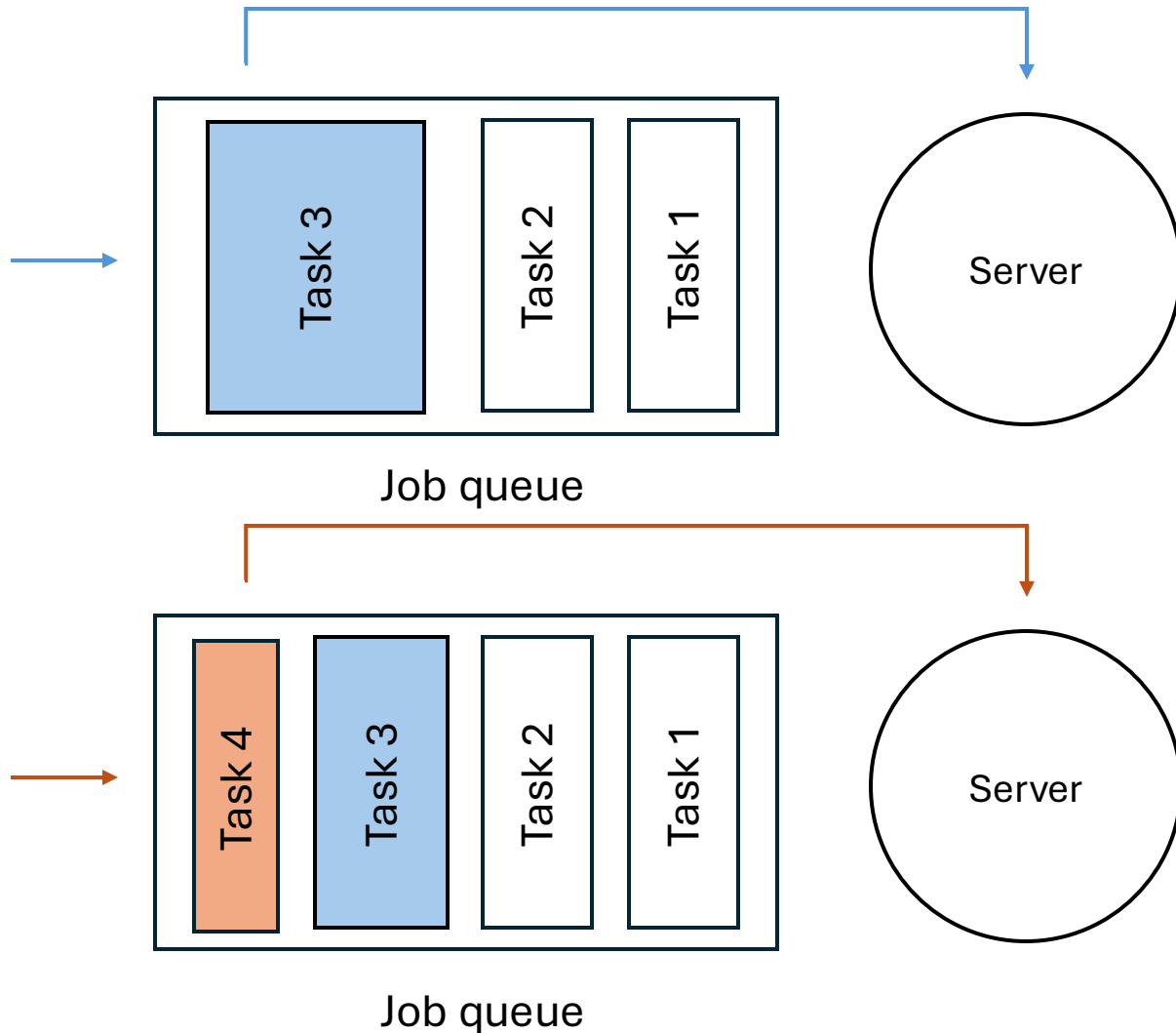
A non-preemptive scheduling strategy that gives priority to jobs based on their length, favoring the shortest ones

Last In First Out (LIFO)



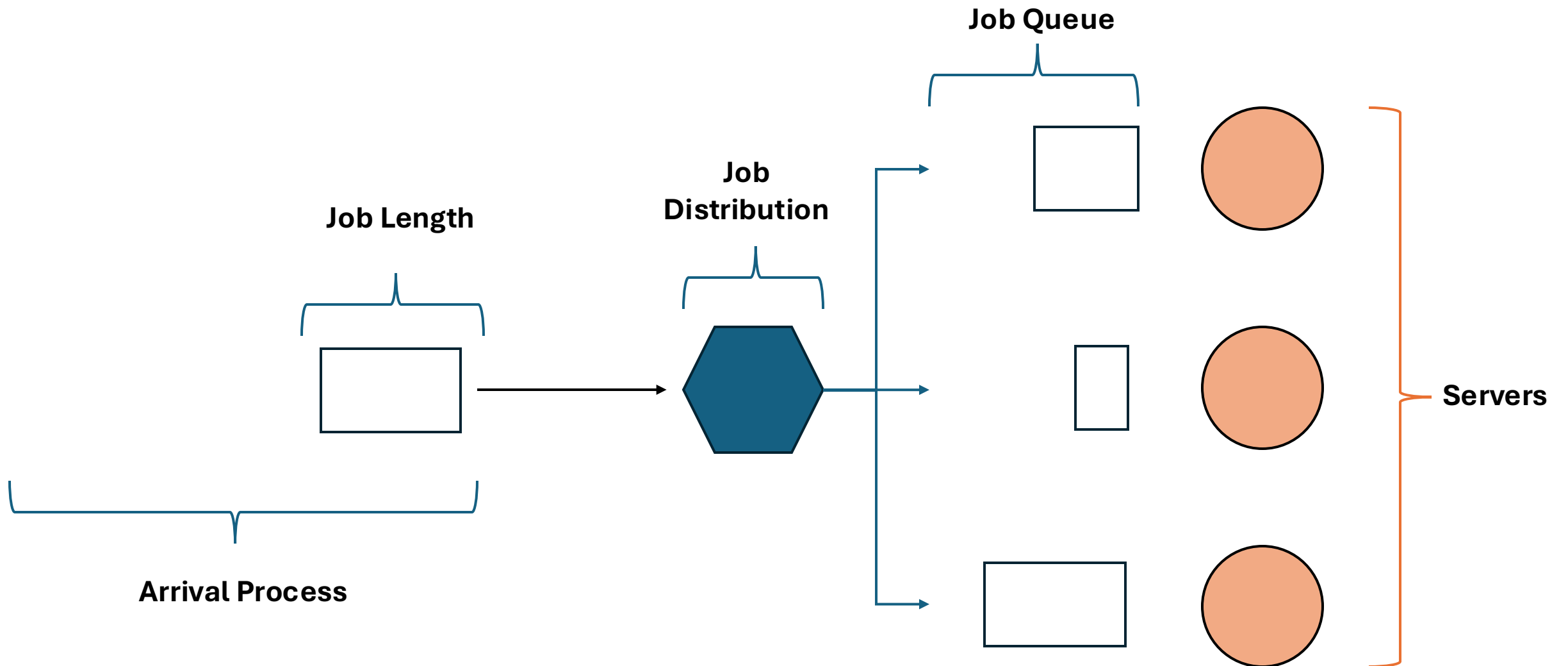
A non-preemptive scheduling strategy that gives priority to jobs based on their arrival order, favoring those which arrive last.

Preemptive Last In First Out (P-LIFO)



A preemptive scheduling strategy that gives priority to jobs based on their arrival order, favoring those which arrive last and allowing them to interrupt ongoing jobs.

Simulating the Scheduling Problem



Evaluation metrics

Metric	Formula	Description
Utilizations	$U_i = \frac{B_i}{T}$	Fraction of time server i was busy during total time T
Mean Latency	$\frac{1}{J} \sum_{j=1}^J (C_j - A_j)$	A_j is the arrival time and C_j the completion time of job j
Throughput	$\frac{J}{T}$	Average number of jobs completed per unit time
CV of Utilizations	$\frac{\sigma_U}{\frac{1}{N} \sum_{i=1}^N U_i}$	Measures load imbalance

Results for arrival rate $\lambda = 1.5$

Exp(1)

Strategy	ML	Th	U_i	CV_U
FIFO	1.51	1.16	[0.63, 0.48, 0.29]	0.30
SJF	1.45	1.16	[0.63, 0.48, 0.29]	0.30
LIFO	1.66	1.16	[0.63, 0.50, 0.26]	0.32
P-LIFO	1.45	1.16	[0.63, 0.48, 0.29]	0.30

LogNorm(0,0.5)

Strategy	ML	Th	U_i	CV_U
FIFO	1.36	0.80	[0.51, 0.24, 0.18]	0.47
SJF	1.36	0.80	[0.51, 0.24, 0.18]	0.47
LIFO	1.42	0.80	[0.51, 0.24, 0.18]	0.47
P-LIFO	1.64	0.80	[0.51, 0.24, 0.18]	0.47

Results for arrival rate $\lambda = 3.5$

Exp(1)

Strategy	ML	Th	U_i	CV_U
FIFO	3.65	2.36	[0.82, 0.79, 0.73]	0.05
SJF	2.40	2.60	[0.83, 0.79, 0.79]	0.02
LIFO	2.39	1.96	[0.84, 0.84, 0.79]	0.03
P-LIFO	1.32	1.96	[0.81, 0.85, 0.78]	0.03

LogNorm(0,0,5)

Strategy	ML	Th	U_i	CV_U
FIFO	3.29	2.08	[0.92, 0.75, 0.78]	0.09
SJF	2.46	2.16	[0.90, 0.80, 0.77]	0.07
LIFO	2.43	2.08	[0.88, 0.80, 0.79]	0.05
P-LIFO	2.42	1.68	[0.90, 0.82, 0.79]	0.05

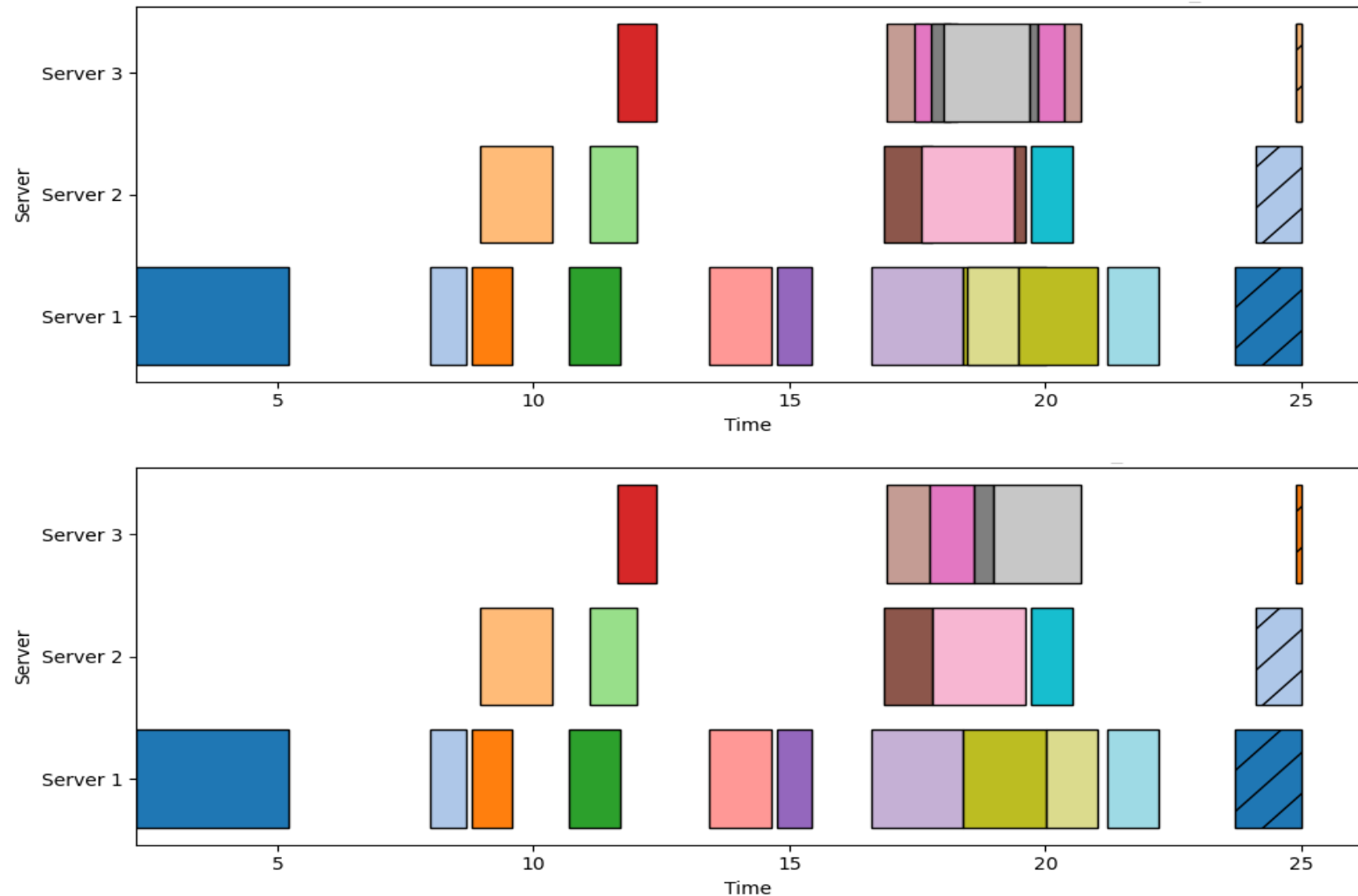
Gantt plots – Low arrival rate – LogNormal Service

P-LIFO

ML	Th	U_i	CV_U
1.64	0.80	[0.51, 0.24, 0.18]	0.47

SJF

ML	Th	U_i	CV_U
1.36	0.80	[0.51, 0.24, 0.18]	0.47



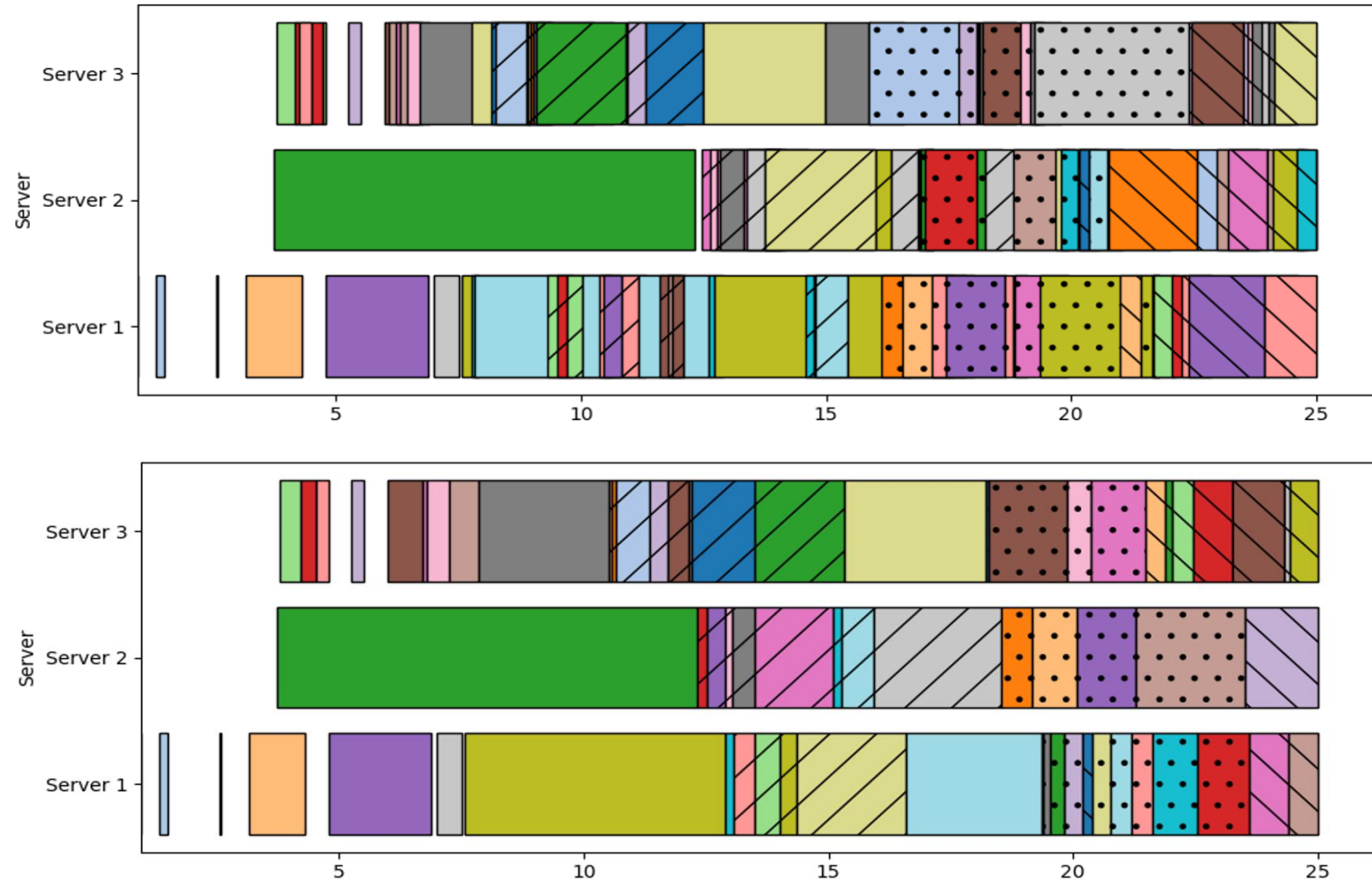
Gantt plots – High arrival rate – Exponential Service

P-LIFO

ML	Th	U_i	CV_U
1.32	1.96	[0.81, 0.85, 0.78]	0.03

SJF

ML	Th	U_i	CV_U
2.40	2.60	[0.83, 0.79, 0.79]	0.02

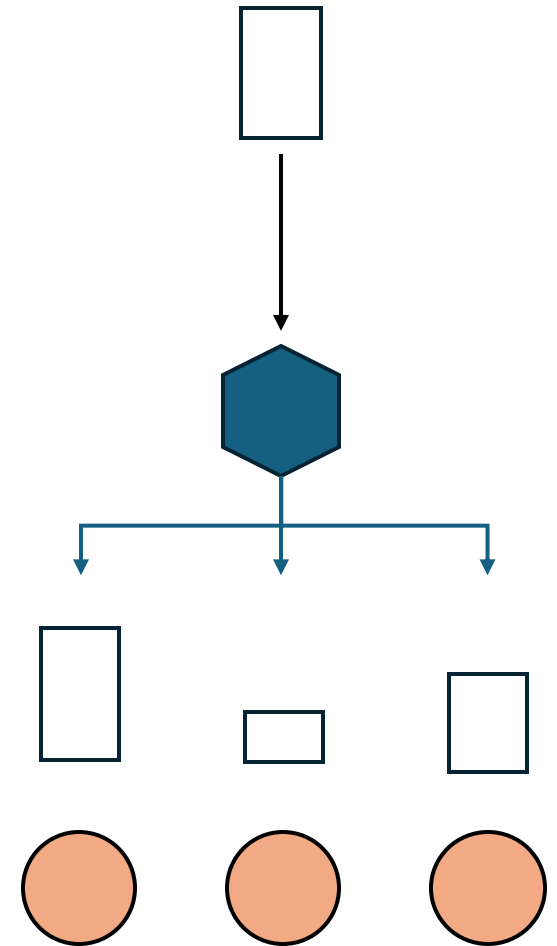


Conclusion

Under low load, all strategies achieved similar throughput and utilization, indicating no queue buildup. In this setting, SJF consistently delivered the lowest latency by prioritizing short jobs when server capacity was available.

Under high load, performance differences became more evident. SJF maintained the highest throughput with relatively low latency. P-LIFO significantly reduced latency when service times were exponential, benefiting from its preempting ability.

In conclusion, no strategy is strictly better than the other, it depends on the setting and what we want to optimize



References

- Project code: github.com/NicolaCortinovis/MAS_2025
- Harchol-Balter, M. (2013). Performance Modeling and Design of Computer Systems. Cambridge University Press
- SimPy library: <https://simpy.readthedocs.io/en/latest>
- Clark, W., Polakov, W. N., \& Trabold, F. W. (1922). The Gantt chart: A working tool of management. New York: The Ronald Press Company

Thank you for the attention!